

OS LAB

Practical-10

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ cd ~/OSLab/file_io
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ echo "Hello Linux inode" > inode_test.txt
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ cat inode_test.txt
Hello Linux inode
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ls -li inode_test.txt
2809 inode_test.txt
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ stat inode_test.txt
  File: inode_test.txt
  Size: 18          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 2809         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/  lahari)   Gid: ( 1000/  lahari)
Access: 2026-09-26 16:13:48.912383736 +0000
Modify: 2026-09-26 16:13:41.791161726 +0000
Change: 2026-09-26 16:13:41.791161726 +0000
 Birth: 2026-09-26 16:13:41.791161726 +0000
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ln inode_test.txt hard_link.txt
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ls -li inode_test.txt hard_link.txt
2809 -rw-r--r--  2 lahari lahari 18 Sep 26 16:13 hard_link.txt
2809 -rw-r--r--  2 lahari lahari 18 Sep 26 16:13 inode_test.txt
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ln -s inode_test.txt symbolic_link.txt
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ls -li inode_test.txt hard_link.txt symbolic_link.txt
2809 -rw-r--r--  2 lahari lahari 18 Sep 26 16:13 hard_link.txt
2809 -rw-r--r--  2 lahari lahari 18 Sep 26 16:13 inode_test.txt
21145 lrwxrwxrwx  1 lahari lahari 14 Sep 26 16:14 symbolic_link.txt -> inode_test.txt
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ls -li inode_test.txt
2809 inode_test.txt
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ find . -inum 123456
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ echo "Modified through original file" > inode_test.txt
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ cat hard_link.txt
Modified through original file
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ cat symbolic_link.txt
Modified through original file
```

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>
```

```
int main() {
    int fd;
    struct stat file_info;
    char *mapped_file;
```

```

// Open the file
fd = open("mmap_test.txt", O_RDWR);

if (fd == -1) {
    perror("Error opening file");
    return 1;
}

// Get file information
if (fstat(fd, &file_info) == -1) {
    perror("fstat error");
    close(fd);
    return 1;
}

printf("File size: %ld bytes\n", file_info.st_size);

// Map the file into memory
mapped_file = mmap(
    NULL,
    file_info.st_size,
    PROT_READ | PROT_WRITE,
    MAP_SHARED,
    fd,
    0
);

if (mapped_file == MAP_FAILED) {
    perror("mmap error");
    close(fd);
    return 1;
}

// Read file through memory
printf("Original content:\n%s\n", mapped_file);

// Modify file through memory
strcpy(mapped_file, "Content modified using mmap()!\n");

// Make sure changes are written to the file
if (msync(mapped_file, file_info.st_size, MS_SYNC) == -1) {
    perror("msync error");
}

```

```
printf("File modified successfully using mmap().\n");

// Unmap the file
munmap(mapped_file, file_info.st_size);

close(fd);

return 0;
}
```

```
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ nano mmap_file.c
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ echo "Original content of the file." > mmap_test.txt
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ cat mmap_test.txt
Original content of the file.
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ gcc mmap_file.c -o mmap_file
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./mmap_file
File size: 30 bytes
Original content:
Original content of the file.

File modified successfully using mmap().
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ cat mmap_test.txt
Content modified using mmap()!lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ |
```